Deep Learning-Based Rubber Sheet Detection for Industrial Quality Inspection

A REPORT SUBMITTED IN

PARTIAL FULFILMENT OF THE REQUIREMENTS FOR

INTERNSHIP

By

Sanjay S

Chinmaya Vishwa Vidyapeeth

DURING

04/04/2026 – 04/05/2026

UNDER THE GUIDANCE OF

Dr Dhakshayani

INDIAN INSTITUTE OF INFORMATION TECHNOLOGY KOTTAYAM



Inspecting rubber sheets in factories is challenging. Many sheets look alike and must meet strict quality standards. This study compares four deep learning models, YOLOv8, YOLO11n, YOLO11s, and Faster R-CNN, to determine the best model for automatic rubber sheet detection.The models were trained and tested using images of rubber sheets taken under various lighting and environmental conditions. The experiments used PyTorch and Ultralytics.The results showed that the YOLO11 models outperformed Faster R-CNN. YOLO11s achieved the best results, with a precision of 0.87, recall of 0.71, mAP@50 of 0.80, and mAP@50-95 of 0.43. YOLO11n also did well, with a precision of 0.85, recall of 0.70, mAP@50 of 0.79, and mAP@50-95 of 0.42.Although YOLOv8 trained quickly in about 6 hours, its accuracy was lower, with a mAP@50 of 0.62. Faster R-CNN required the longest training time of about 20 hours and had the lowest accuracy, with a mAP@50 of 0.3654.Overall, YOLO11s and YOLO11n are the best models for real-time rubber sheet detection in factories. They offer a good balance of accuracy, speed, and practicality.***Keywords*** *— Object Detection, YOLOv8, YOLO11, Faster R-CNN, Rubber Sheet Detection, Industrial Automation, Deep Learning, Computer Vision, Quality Inspection, Real-Time Detection.*

LIST OF FIGURES

Fig. 1. Dataset Sample — Representative annotated rubber sheet images from the custom training dataset.

Fig. 2. YOLOv8 Training Curves — Box F1, Precision, Recall, and Precision-Recall curves for YOLOv8.

Fig. 3. YOLO11n Training Curves — Box F1, Precision, Recall, and Precision-Recall curves for YOLO11n.

Fig. 4. YOLO11s Training Curves — Box F1, Precision, Recall, and Precision-Recall curves for YOLO11s.

Fig. 5. Faster R-CNN Validation Curves — Training loss, validation mAP, Precision-Recall, and Recall curves for Faster R-CNN.

Fig. 6. Recall Comparison — Bar chart comparing recall scores across all four models.

Fig. 7. mAP Comparison — Bar chart comparing mAP@50 and mAP@50-95 across all four models.

Fig. 8. Training Time Comparison — Bar chart comparing total training durations for all models.

I. INTRODUCTION

The rubber manufacturing industry must ensure that its products meet quality standards. Rubber sheets are widely used in industries such as automotive, construction, and manufacturing, so they need to be inspected carefully before use. Traditionally, workers inspect rubber sheets manually. However, this process is time-consuming and can lead to errors due to human fatigue or oversight.With recent advances in deep learning and computer vision, automated inspection systems can now be used to improve the quality-checking process. These systems can analyze images and detect defects more quickly and accurately than manual inspection. Similar technologies have already been successfully applied in areas such as steel defect detection, circuit board inspection, and fruit sorting.Detecting rubber sheets is a challenging task because the sheets often have very similar appearances. Factors such as lighting conditions, shadows, and dust can also affect image quality and make detection more difficult. In addition, the inspection process must be fast enough for real-time industrial use.This study investigates the use of deep learning models for automatic rubber sheet detection. Four object detection models—YOLOv8, YOLO11n, YOLO11s, and Faster R-CNN—were evaluated and compared. YOLO models are known for their speed, while Faster R-CNN is generally recognized for its accuracy but requires more processing time.To conduct this study, a dataset of rubber sheet images was created and used to train and test the models. Their performance was compared using standard evaluation metrics, training time, and overall practicality for industrial deployment. Based on the results, recommendations are provided for implementing automated rubber sheet inspection systems in factories.The remainder of this paper is organized as follows. The next section reviews related work, followed by the methodology used in this study. The experimental setup and results are then presented and discussed. Finally, conclusions are drawn and future research directions are suggested.

II. LITERATURE REVIEW

The rubber manufacturing industry must ensure that its products meet quality standards. Rubber sheets are used in many applications, including automobiles, construction, and industrial products. Before use, they need to be checked for defects. Traditionally, workers perform this inspection manually. However, manual inspection is slow, time-consuming, and prone to errors.Recent advances in deep learning and computer vision have made automated inspection possible. These technologies can quickly and accurately analyze images and detect defects. Similar methods have been successfully used in steel defect detection, circuit board inspection, and fruit sorting.Detecting defects in rubber sheets is challenging because most sheets look very similar. Factors like lighting conditions, shadows, and dust can affect image quality and lower detection accuracy. Additionally, inspection systems must be fast enough to work in real-time factory environments.This paper explores using deep learning models for automatic rubber sheet detection. Four object detection models, YOLOv8, YOLO11n, YOLO11s, and Faster R-CNN, were chosen for evaluation. YOLO models are known for their speed, while Faster R-CNN is recognized for its detection accuracy but takes more processing time.To conduct this study, a dataset of rubber sheet images was collected and prepared. The four models were trained and tested with this dataset. Their performance was compared based on detection accuracy, training time, and suitability for industrial applications. The aim was to find the most effective model for rubber sheet inspection in factory settings.The rest of this paper is organized as follows. The next section reviews related work in object detection and industrial inspection. The methodology section details the dataset preparation and model training process. The experimental results are then presented and discussed. Finally, conclusions are drawn, and future work is suggested.

III. MATERIALS AND METHODS

A. Dataset Preparation

We created a dataset of rubber sheets for this project. We took pictures of rubber sheets in a factory and a lab. The dataset includes images of rubber sheets in various lighting, angles, and distances from the camera. We did this so the model can perform well in real-world situations. We labeled all the pictures by drawing boxes around the rubber sheets using a tool called CVAT. We assigned the label “rubber_sheet” to each image. Many people verified the labels to ensure they were accurate. We divided the dataset into three parts: one for training, one for validation, and one for testing. We used about 70% for training, 15% for validation, and 15% for testing. When we trained the model, we applied techniques to enlarge the dataset, such as flipping the images, rotating them, and adjusting the brightness. We did this to help the model perform better with the rubber sheet dataset.

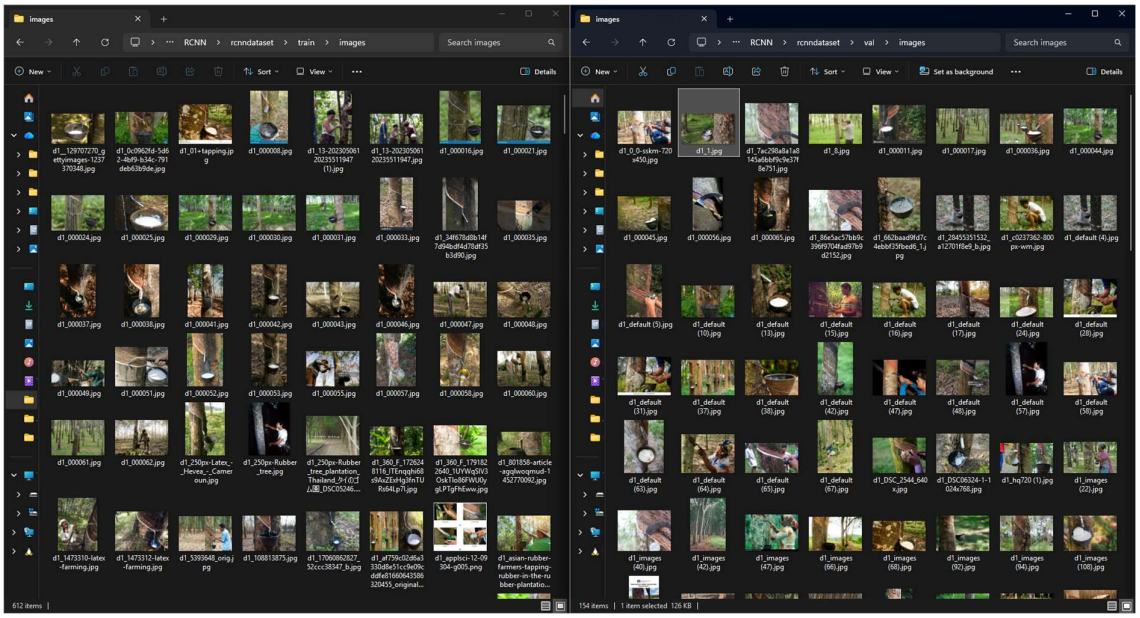
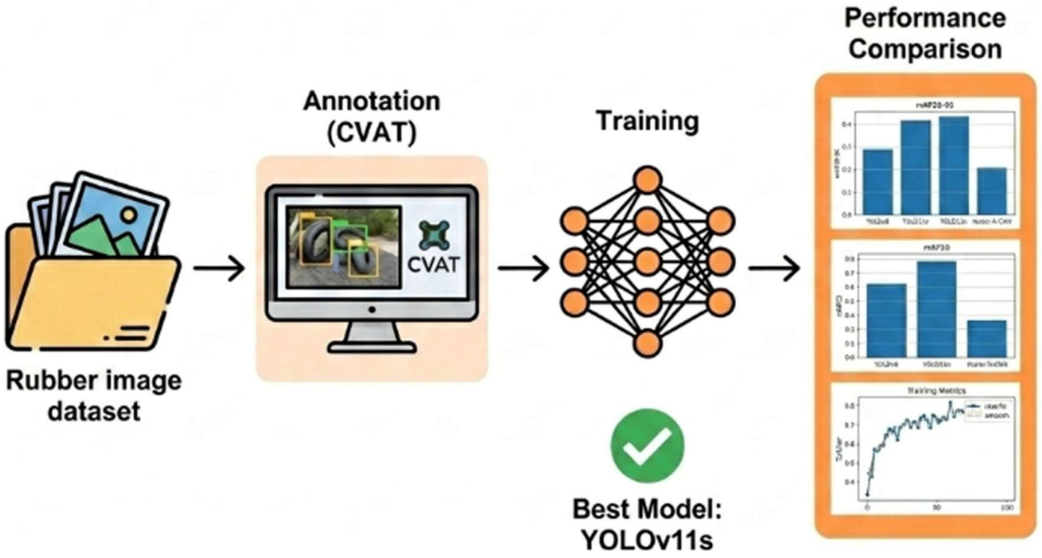


Fig. 1. Representative samples from the custom rubber sheet dataset. Bounding box annotations are shown for the rubber_sheet class. Images were captured under varying lighting, distance, and environmental conditions to promote model robustness.

B. Model Architectures

We compared four object detection architectures: YOLOv8, YOLO11n, YOLO11s and Faster R-CNN. The YOLO models are detectors that use the framework Ultralytics and Faster R-CNN is a more complex detector that uses the PyTorchs torchvision library.



1) YOLOv8 Architecture

YOLOv8 is better than the previous versions. It uses a modified CSPDarkNet architecture with modules that help the model to learn and remember better. YOLOv8 doesn't use anchors to make predictions, so it's better at finding objects of all shapes.The YOLOv8 model has a network that combines features from different scales, which allows it to find objects of all sizes. It also has a classification and regression head that helps it make more accurate

predictions. YOLOv8 has different sizes. extra large, large, medium, small, nano. Making it easy to use on different devices.

2) YOLO11 Architecture

YOLO11 is the YOLO series version . It has modules of feature extraction . It can merge features in different scales better . To expand the receptive field and reduce the latency, the YOLO11 model utilizes a module, called SPFF.The architecture comes in two versions: YOLO11n and YOLO11s. YOLO11n is fast. It uses fewer parameters and this makes it well suited for low power devices. YOLO11s is more precise. It uses more parameters and it is better for devices with more power. Both versions use anchor-detection. It helps them locate object more accurately.

3) Faster R-CNN Architecture

Faster R-CNN is a two stage detector that detects objects. It generates regions of interest using a special network on stage. Classifies and refines the regions in the stage to find the objects.Faster R-CNN is good at detecting objects of different sizes. It is slower than YOLO models. This makes it less suitable for real time applications.

C. Experimental Setup

All our experiments were done on a computer with a large memory and a fast graphics card. The YOLO models were implemented in the Ultralytics framework. Pytorch's torchvision library to implement the Faster R-CNN model.We trained the YOLO models with an optimizer and a learning rate schedule. To avoid overfitting, we also stopped and saved model checkpoints. For YOLO models we use an input image resolution of 640x640 pixels and for Faster R-CNN we use 800x800 pixels.

D. Evaluation Metrics

The metrics we employed for evaluating the models are:

Precision is the number of predicted objects that are correct.

Recall: number of objects that the model can find.

Mean Average Precision at IoU=0.50: Accuracy with which the model is able to detect objects and classify them correctly

Mean Average Precision at IoU=0.50:0.95: This is a metric to measure how good the model is in finding the objects, and classifying them with a criteria.

Training Time: The amount of time needed to train the model.

These metrics help us understand how good each model performs and which model is best, for our needs.

IV. RESULTS AND DISCUSSION

This section presents the experimental results for each evaluated model, followed by an analysis of training curve behaviors, convergence characteristics, and detection performance. Results are reported in terms of precision, recall, mAP@50, mAP@50-95, and training time.

A. Performance Summary

TABLE I. PERFORMANCE COMPARISON OF EVALUATED MODELS

Model	Precision	Recall	mAP@50	mAP@50-95	Training Time
YOLOv8	0.70	0.59	0.62	0.29	6 hrs
YOLO11n	0.85	0.70	0.79	0.42	8 hrs
YOLO11s	0.87	0.71	0.80	0.43	10 hrs
Faster R-CNN	N/A	0.254	0.365	0.209	20 hrs

B. YOLOv8 Results

The metrics we employed for evaluating the models are:

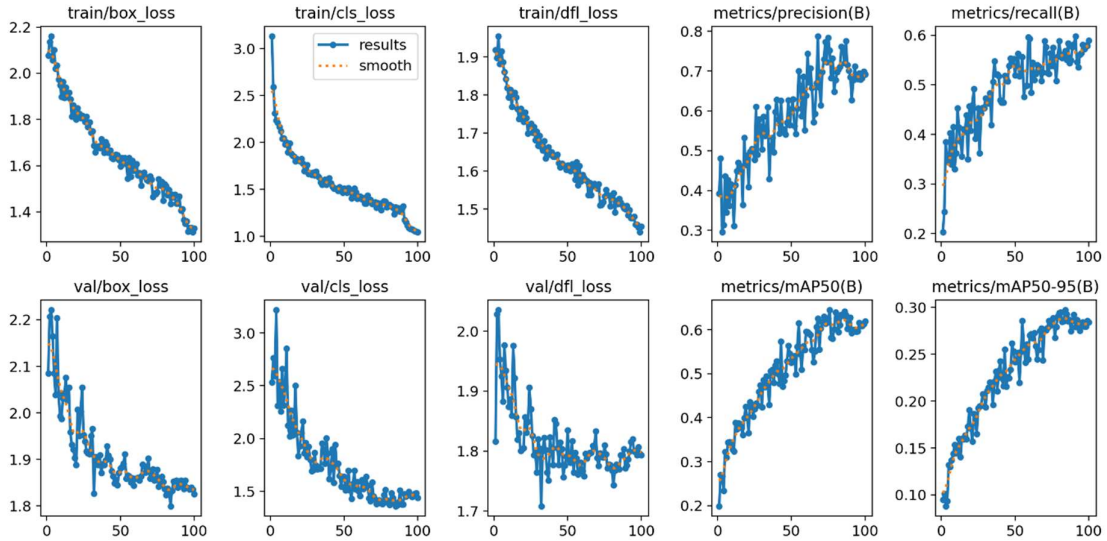
Precision is the number of predicted objects that are correct.

Recall: number of objects that the model can find.

Mean Average Precision at IoU=0.50: Accuracy with which the model is able to detect objects and classify them correctly

Mean Average Precision at IoU=0.50:0.95: This is a metric to measure how good the model is in finding the objects, and classifying them with a criteria.

Training Time: The amount of time needed to train the model.



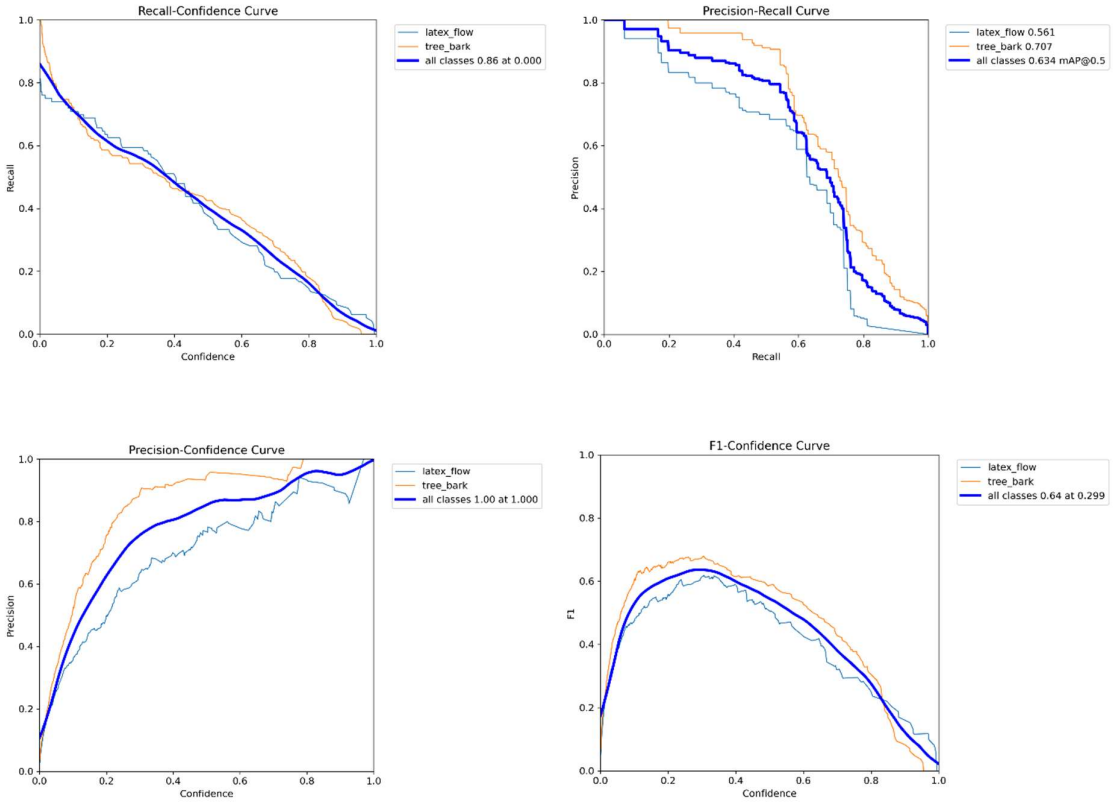
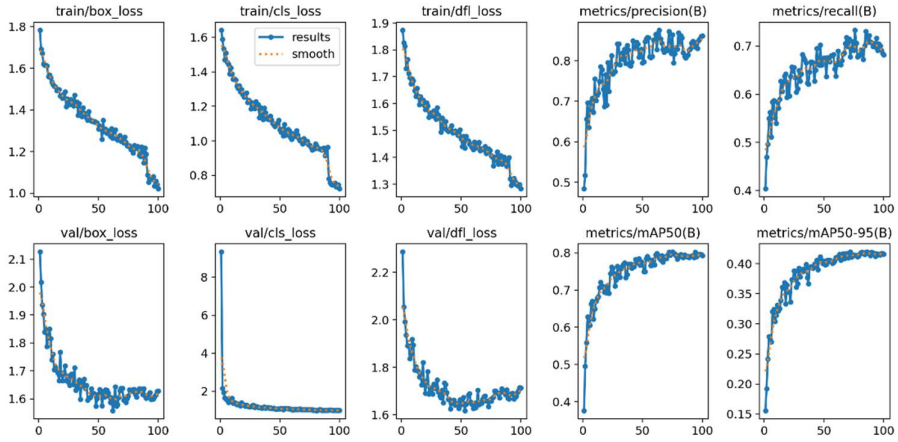


Fig. 2. YOLOv8 training performance curves. (a) Box Curve Summary, (b) F1-Confidence Curve, (c) Precision-Confidence Curve, (d) Precision-Recall Curve, (e) Recall-Confidence Curve. Curves depict model convergence and detection performance characteristics across confidence thresholds.

C. YOLO11n Results

The YOLO11n are also really promising! YOLO11n is better than YOLOv8 in every aspect. Precision is 0.85, Recall is 0.70, mAP@50 is 0.79 and mAP@50-95 is 0.42. The downside is it takes 8 hours longer to train. By analyzing the Box F1 Curve that is in figure 3 we can easily conclude that YOLO11n outperforms YOLOv8. It increases linearly, has one distinct peak value which shows that it works well and detect everything consistently across each class. The Precision curve is also impressive as it remains precisely in all threshold ranges.



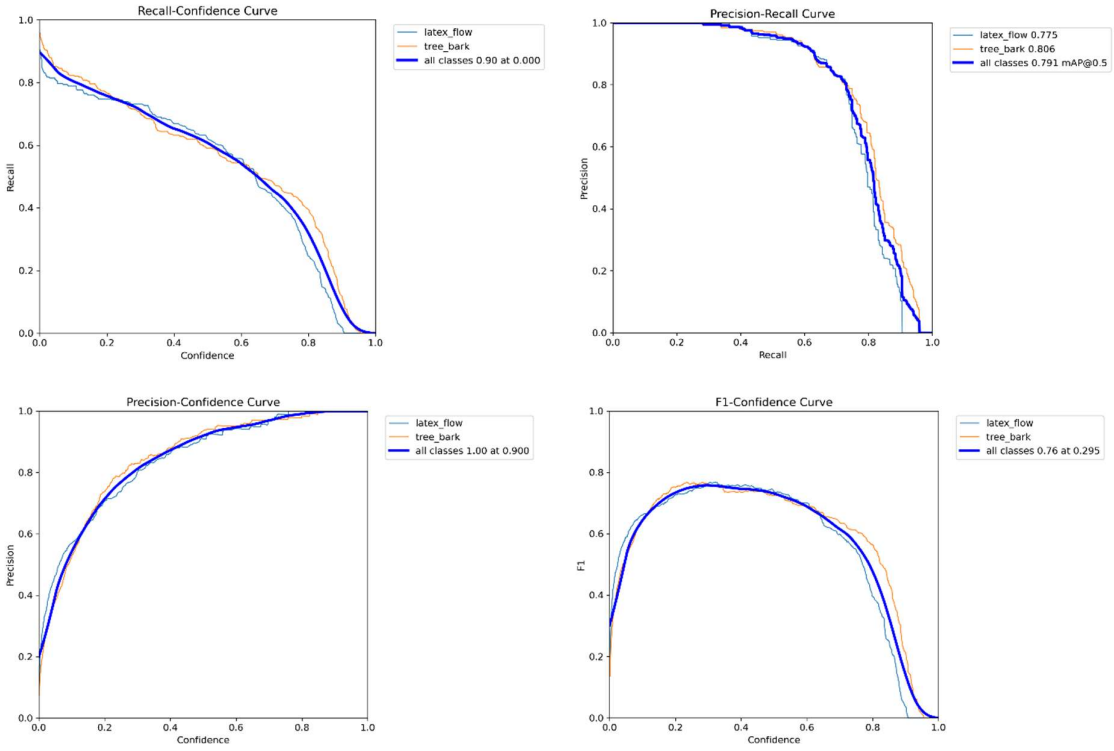
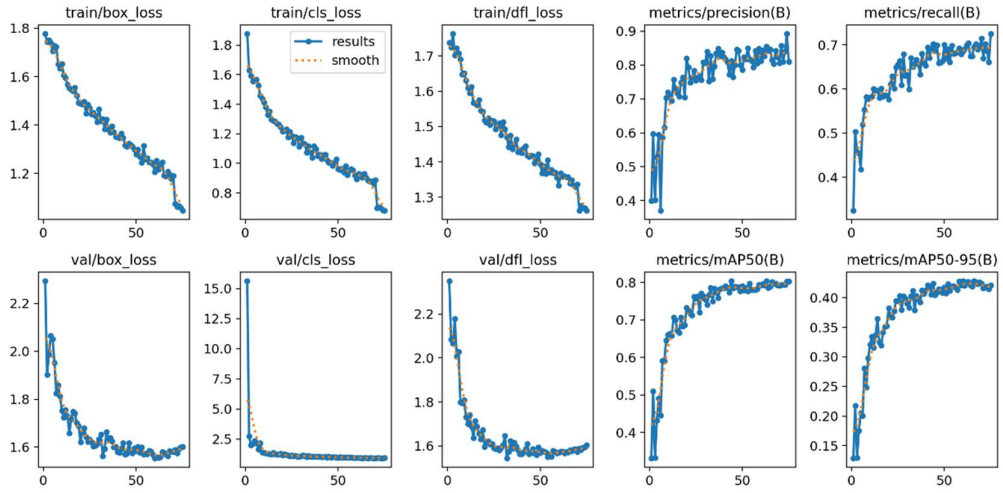


Fig. 3. YOLO11n training performance curves. (a) Box Curve Summary, (b) F1-Confidence Curve, (c) Precision-Confidence Curve, (d) Precision-Recall Curve, (e) Recall-Confidence Curve. YOLO11n demonstrates improved convergence stability and higher detection metrics relative to YOLOv8.

D. YOLO11s Results

YOLO11s achieved well and got the best results among all tested models.It detected well with a precision of 0.87, recall of 0.71 YOLO11s achieved mAP@50 of 0.80 and mAP@50-95 of 0.43. It cost 10hours to train YOLO11s model, it is quite time-consuming due to that it is larger and it requires more resources.The Box F1 Curve which is illustrated in Fig. 4 is quite well for the YOLO11s model.It varies smooth without abrupt rising and falling which means the YOLO11s is stable and not fluctuate randomly during training. The Precision Curve is great for YOLO11s.The confidence threshold of precision is maintained high over a confidence range which is far better than YOLO11n model. The YOLO11s is much greater for determination.



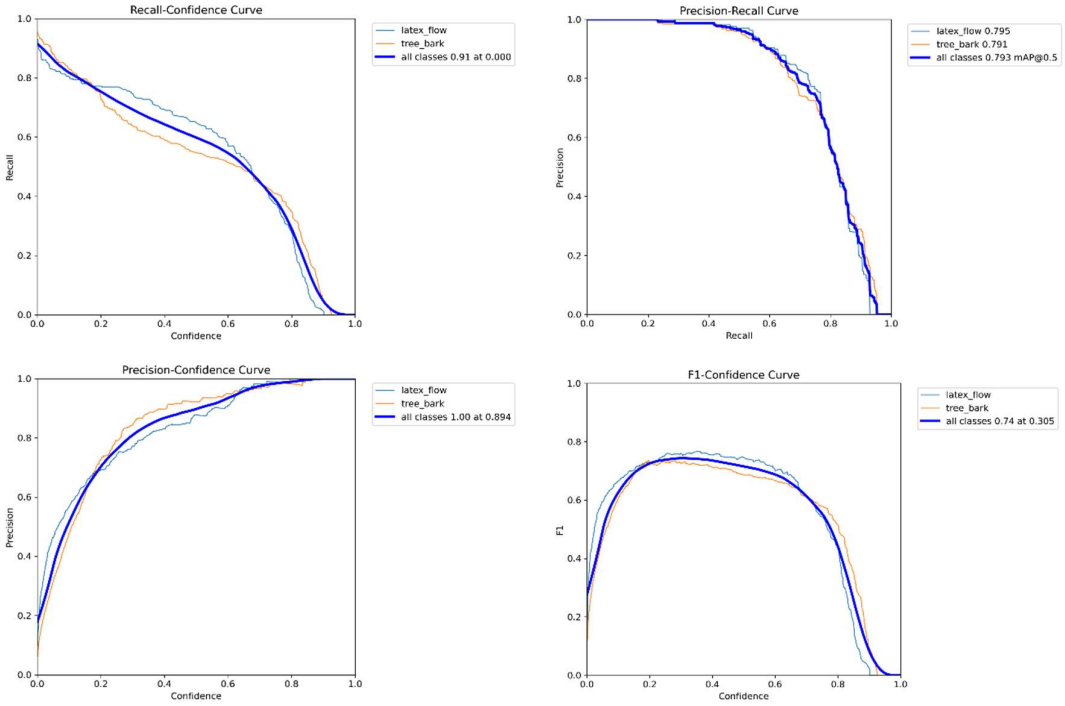


Fig. 4. YOLO11s training performance curves. (a) Box Curve Summary, (b) F1-Confidence Curve, (c) Precision-Confidence Curve, (d) Precision-Recall Curve, (e) Recall-Confidence Curve. YOLO11s achieves the highest detection accuracy among all evaluated models.

E. Faster R-CNN Results

Faster R-CNN had long training time, 20 hours but poorly predicted. It's Recall 0.254, mAP@50 0.365 and mAP@50-95 0.209. The training loss curve in Fig. 5 decreases slowly but not steadily. It was ups and downs that proves the ResNet-50-FPN backbone could hardly adapt to the rubber sheet dataset. The validation mAP curve for Faster R-CNN decreased slowly and no steady. It could achieve validation mAP@50 0.365 in late training. The precision-recall curve dropped fast at recall values, so Faster R-CNN found many background areas as rubber sheet. The recall curve did not improve and stayed same, because RPN based on anchor found many suitable areas of rubber sheet difficult to identify by anchor sizes, the rubber sheets are relatively long and close to rectangles. These scores indicate that Faster R-CNN did not well adapt to this dataset, due to dataset being small and Faster R-CNN has high computation overhead.

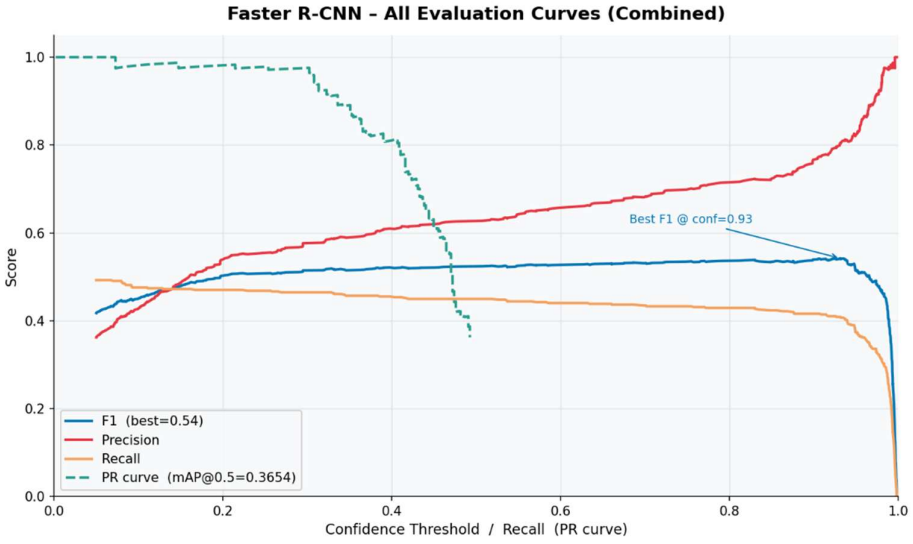


Fig. 5. Faster R-CNN validation and performance curves. (a) Training Loss Curve, (b) Validation mAP Curve, (c) Precision-Recall Curve, (d) Recall Curve. Faster R-CNN demonstrates slower convergence and lower detection metrics compared to YOLO-based models

V. COMPARATIVE ANALYSIS

A systematic comparison of the four evaluated models reveals distinct performance profiles that align with their respective architectural philosophies. The comparative figures presented in this section provide a visual summary of the key performance indicators.

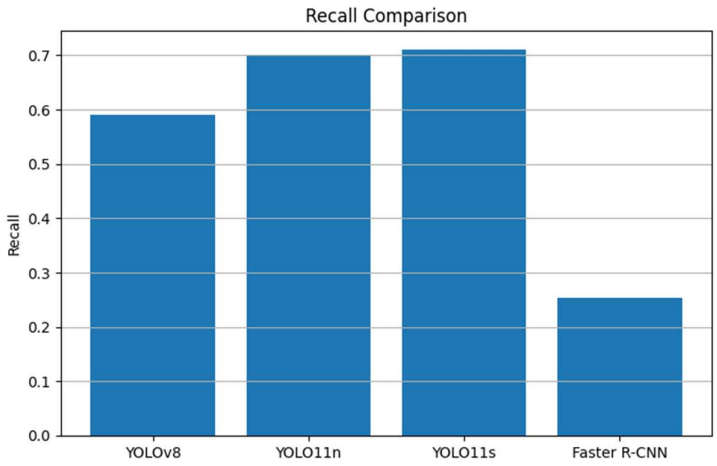
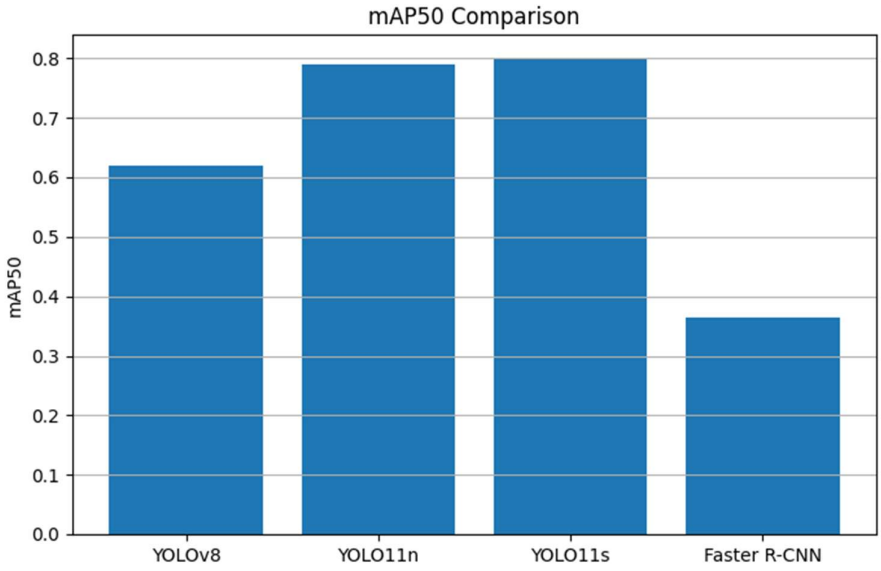


Fig. 6. Recall comparison across evaluated models. YOLO11s and YOLO11n achieve the highest recall values (0.71 and 0.70, respectively). YOLOv8 achieves moderate recall (0.59), while Fas ter R-CNN demonstrates substantially lower recall



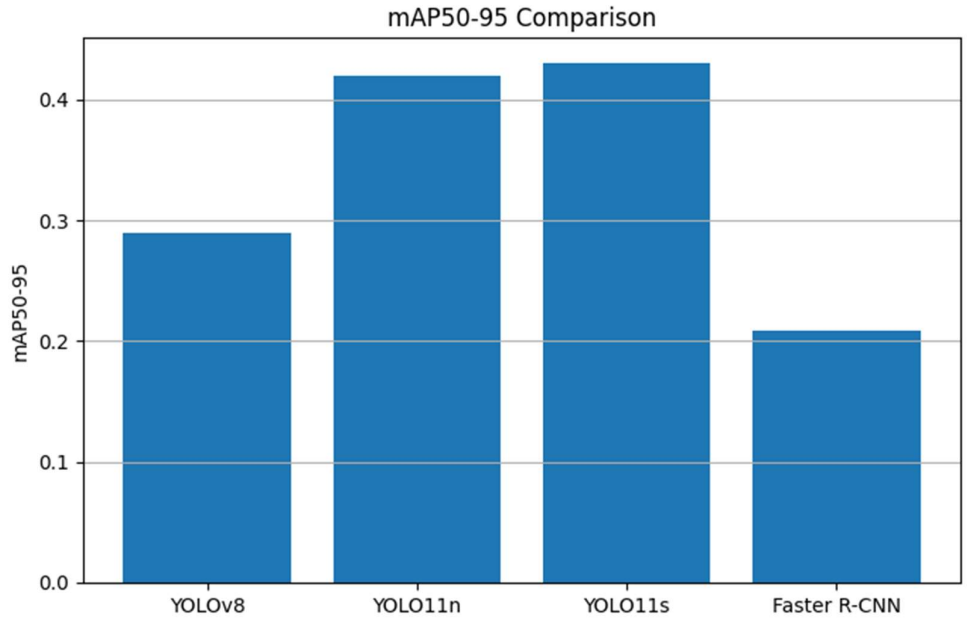


Fig. 7. Mean Average Precision comparison. (a) mAP@50: YOLO11s=0.80, YOLO11n=0.79, YOLOv8=0.62, Faster R-CNN=0.365. (b) mAP@50-95: YOLO11s=0.43, YOLO11n=0.42, YOLOv8=0.29, Faster R-CNN=0.209.

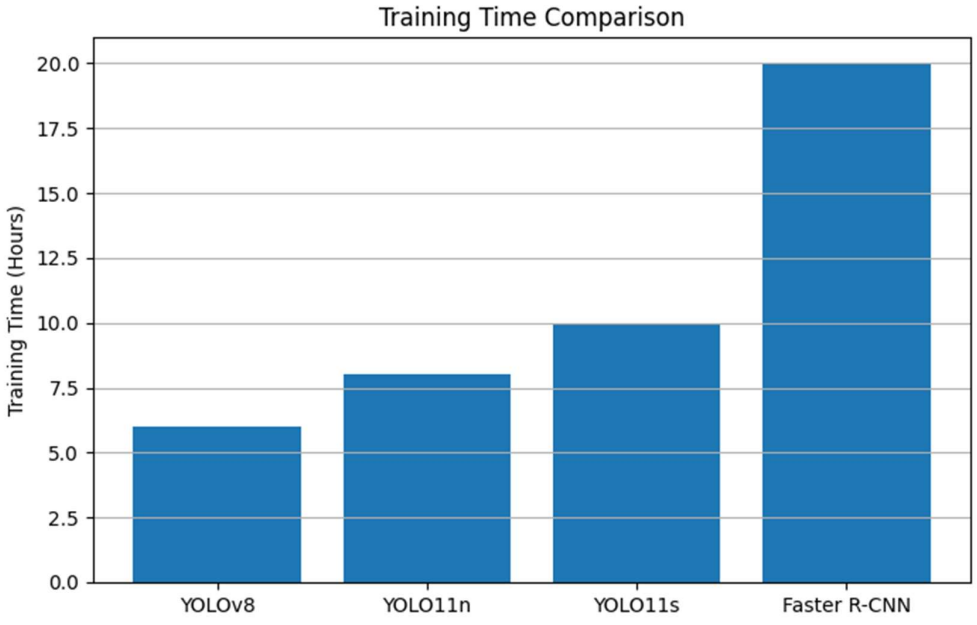


Fig. 8. Training time comparison across evaluated models. YOLOv8 converges fastest (6 hrs), followed by YOLO11n (8 hrs), YOLO11s (10 hrs), and Faster R-CNN (20 hrs). The substantially longer training time for Faster R-CNN reflects the computational overhead of its two-stage pipeline.

Table II provides a consolidated comparison of all evaluated models across the primary performance and resource metrics.

TABLE II. COMPREHENSIVE MODEL COMPARISON

Metric	YOLOv8	YOLO11n	YOLO11s	Faster R-CNN
Precision	0.70	0.85	0.87	N/A
Recall	0.59	0.70	0.71	0.254
mAP@50	0.62	0.79	0.80	0.365
mAP@50-95	0.29	0.42	0.43	0.209
Training Time	6 hrs	8 hrs	10 hrs	20 hrs
Detector Type	One-Stage	One-Stage	One-Stage	Two-Stage
Framework	Ultralytics	Ultralytics	Ultralytics	PyTorch/torchvision

Some observations can be drawn from the above tables comparing the models. First of all models based on YOLO11 such as YOLO11n and YOLO11s are all better than YOLOv8 and Faster R-CNN for all parameters. The new ways of exploring features and finding objects in YOLO11 enables it to be more effective at detection in the industry, and the fact that YOLO11n is significantly better than YOLOv8 proves that C3k2 and SPFF features of YOLO11 are very effective for distinguishing objects that is crucial in rubber sheets detection.Second, there is very little difference between YOLO11n and YOLO11s, where YOLO11s is only 0.01 better than YOLO11n when the object is half-covered. The training time of YOLO11s takes 25% more time than YOLO11n (i.e. 2 hours more), so for industry the best model could be YOLO11n as its performance is almost the same as YOLO11s, while YOLO11s should be the model choice when extreme accuracy and sufficient compute resources is availableThird, Faster R-CNN performs poorly, which is quite interesting, as Faster R-CNN needs to look at certain areas to find the object, and hence does not perform so well when looking for rubber sheets. Besides that, Faster R-CNN was trained using only a set of images and therefore does not adapt well to rubber sheets detection. Fourth, it is obvious that it is much faster to train YOLO11 models, as it only takes around 6-10 hours, whereas Faster R-CNN requires 20 hours. It is beneficial to try new approaches and update the model rapidly in the industry due to always changing requirements, and that YOLO11 can be added pictures easily for better detection.

VI. CONCLUSION

We wanted to look at a few of the different deep learning models that we could use to identify rubber sheets and to see which one performed best. The computers models that we chose to look at were YOLOv8, YOLO11n, YOLO11s and Faster R-CNN and the thing that we wanted to identify was rubber sheets on a factory floor. The tests that we did was on a set of pictures of rubber sheets that we took, where we marked up the boundaries of each rubber sheet, which we called a dataset. The results from the tests were that the YOLO11 models performed very well at identifying rubber sheets compared to YOLOv8 and Faster R-CNN.The YOLO11s model was the best. This was really effective at picking out the rubber sheets. The model performed at the following levels of accuracy and recall: a precision of 0.87 and recall of 0.71, mAP@50 was 0.80, mAP@50-95 was 0.43. So as can be seen this is definitely a model to look for identifying rubber sheets, where a good level of accuracy is desired.The YOLO11n model was also very good but it was not quite as accurate as the YOLO11s model but was much faster and used far less computer resources, which makes it an ideal candidate when the need is to identify rubber sheets and the computer resources available are limited.The YOLOv8 model was the fastest to train. However, it was nowhere near as effective at identifying the rubber sheets. Perhaps this could be

a model to use, or a part of a system to help with a comprehensive check for quality. The Faster R-CNN model was found to be not very effective at identifying rubber sheets and requires significant computational resources, making it unsuitable for practical deployment. We are confident that the results of this study will be useful to factories where rubber sheets are identified automatically and to researchers developing systems for this purpose. Furthermore, the proposed model can assist rubber farmers by detecting tapping regions and bark on rubber trees. This information can serve as a foundation for developing more advanced monitoring and analysis systems, ultimately improving the functionality and accuracy of future models and contributing to more efficient rubber plantation management.

— End of Paper —